



Student Name	ID
Amal Yahya	202207002
Bushra salman	202206470
Radhya Hasan	202210255

Fraud Detection Using Machine Learning

Regression Models

I. Abstract

This project explores the application of machine learning techniques, particularly regression models such as Linear Regression and Random Forest, to detect fraudulent financial transactions. Using a synthetic fraud dataset, we aim to preprocess the data, train models, and evaluate their performance in classifying fraudulent activities. The expected outcome is a comparison between models in terms of accuracy, interpretability, and effectiveness in identifying fraud.

II. Introduction

Fraudulent financial activities are a major concern in the digital era, causing significant economic losses for individuals, organizations, and governments. Detecting such fraudulent behavior as early as possible is crucial to prevent further damage and secure financial systems.

Machine learning and data science provide effective tools for analyzing large datasets, learning from patterns, and making accurate predictions. In the context of fraud detection, these techniques enable automatic and efficient identification of anomalies that could

indicate fraud.

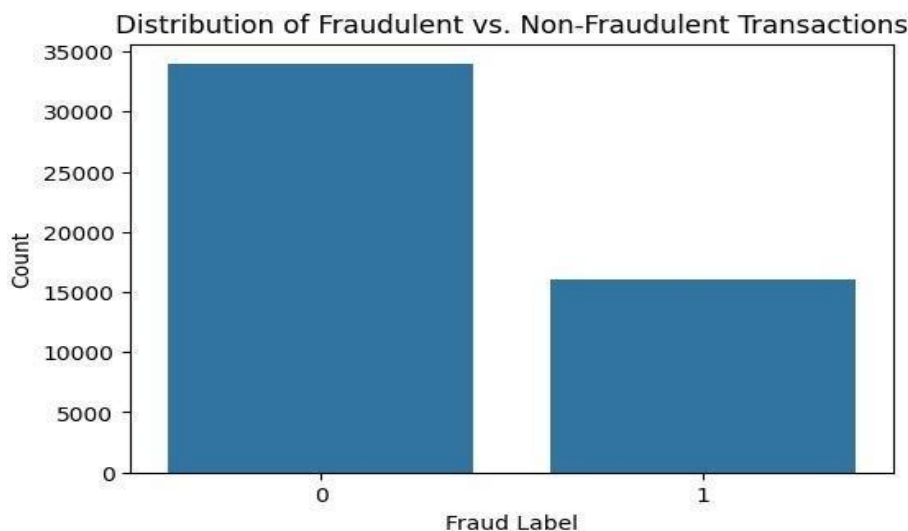
In this study, we utilize machine learning regression models, including Linear Regression and Random Forest, to analyze transaction data and identify fraudulent activities. Our methodology includes data preprocessing, model training, evaluation, and visualization to demonstrate the effectiveness of the proposed approach

III. Description of Dataset and Preprocessing

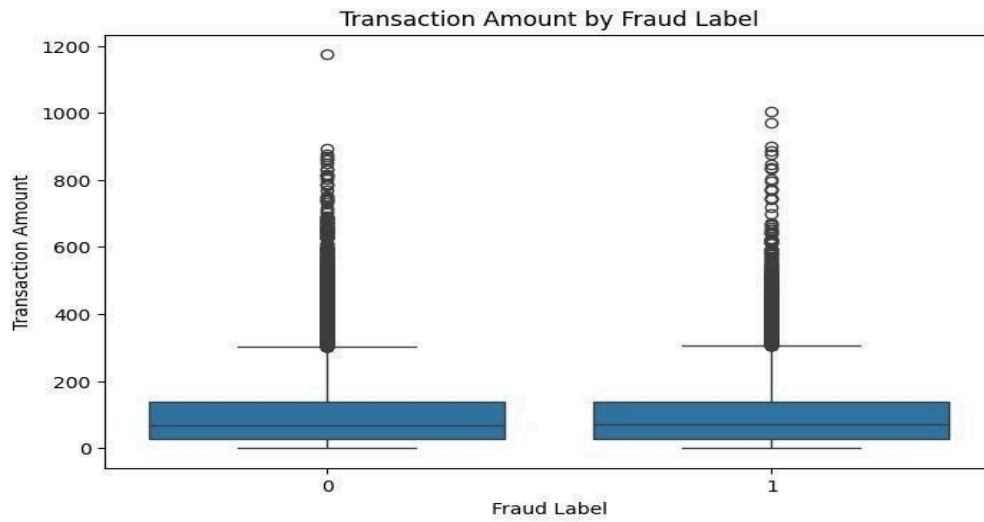
The dataset used is a synthetic fraud detection dataset obtained from Kaggle. It includes features such as transaction type, device type, location, merchant category, transaction amount, time, and fraud label. Categorical features were encoded using Label Encoding, and numerical features were standardized using StandardScaler. Time-based features like Hour and Day of Week were extracted from timestamps.

You can insert the following visualizations here:

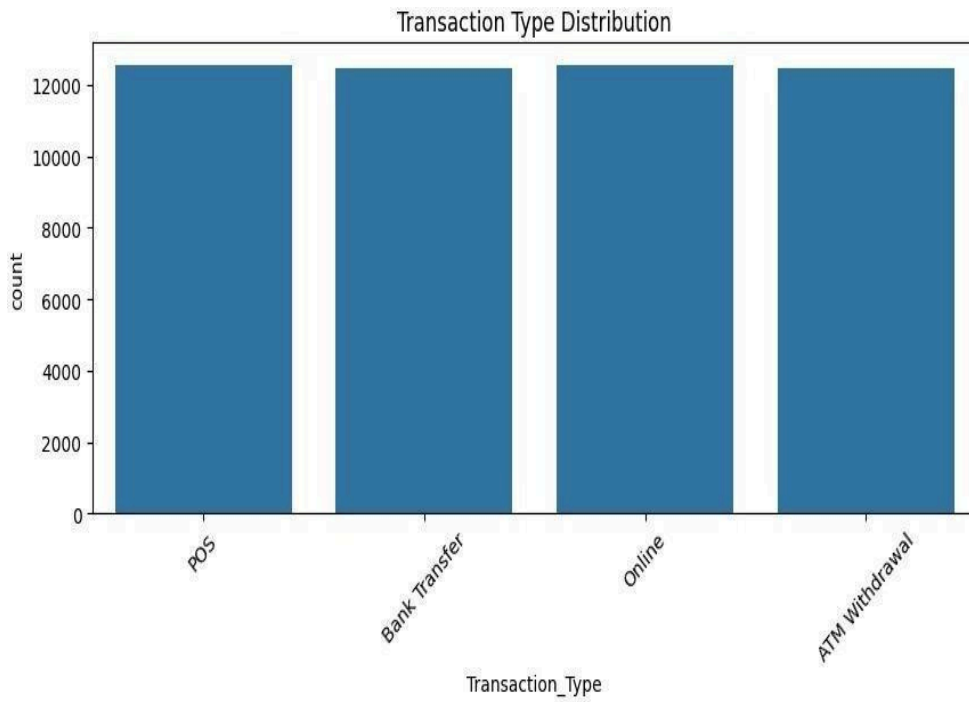
-Distribution of Fraudulent vs. Non-Fraudulent Transactions



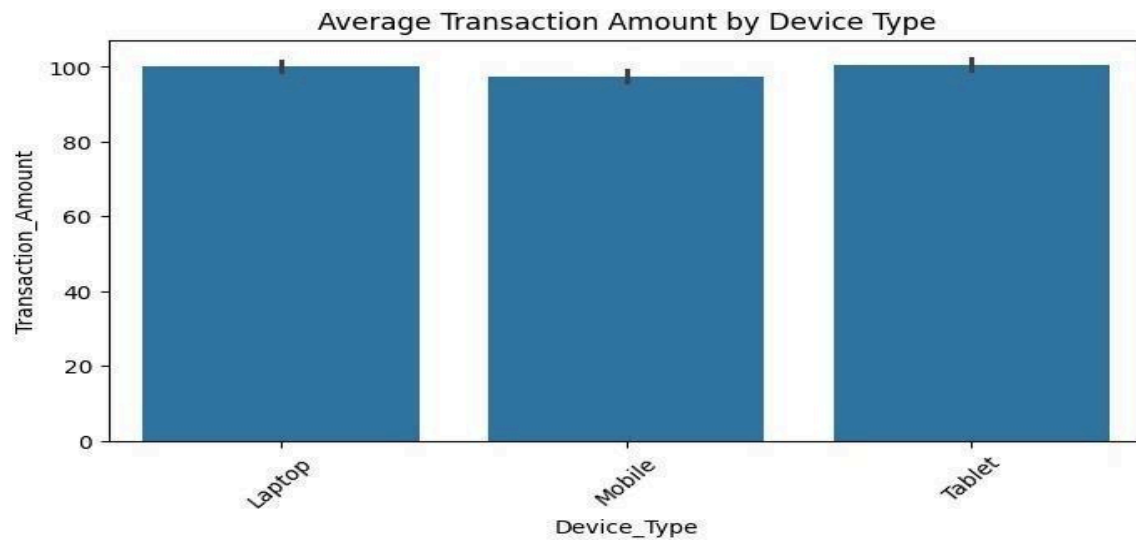
- Transaction Amount by Fraud Label



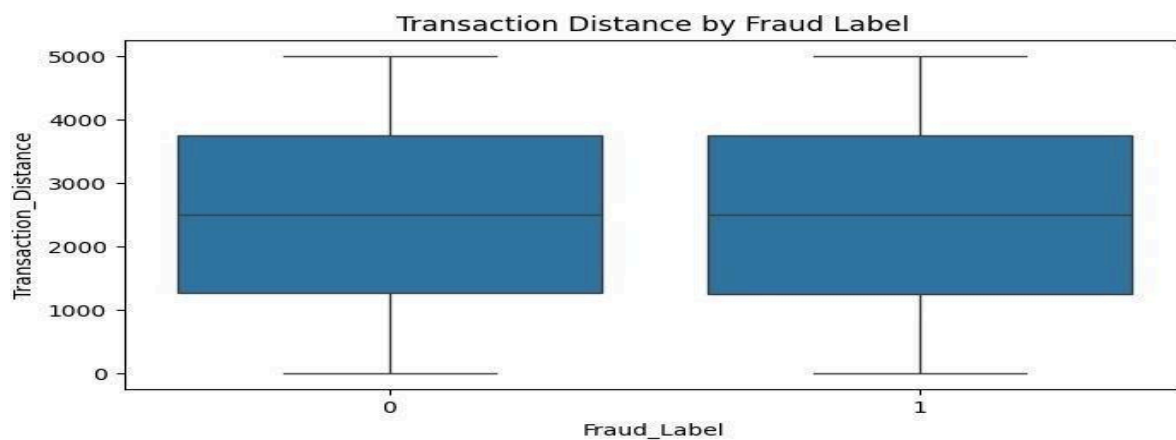
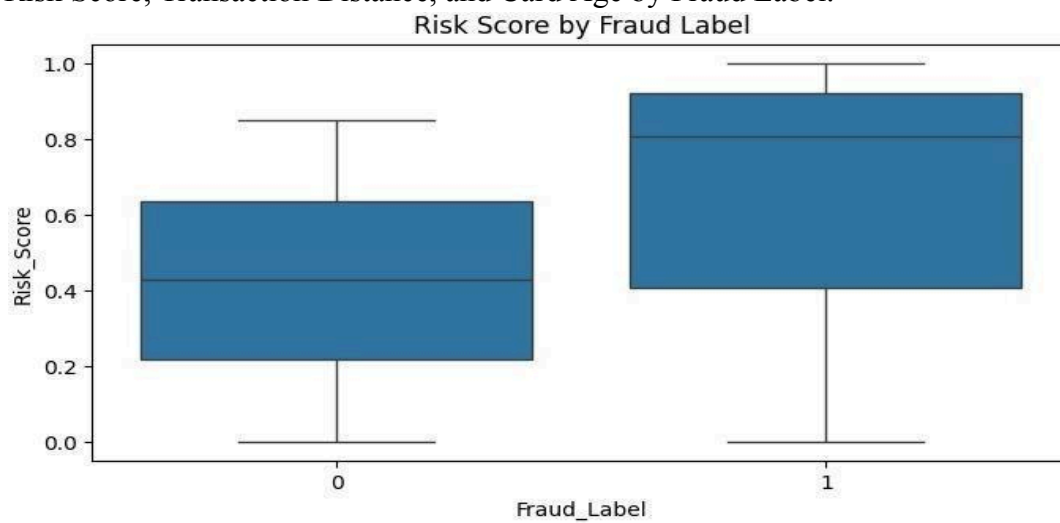
- Transaction Type Distribution

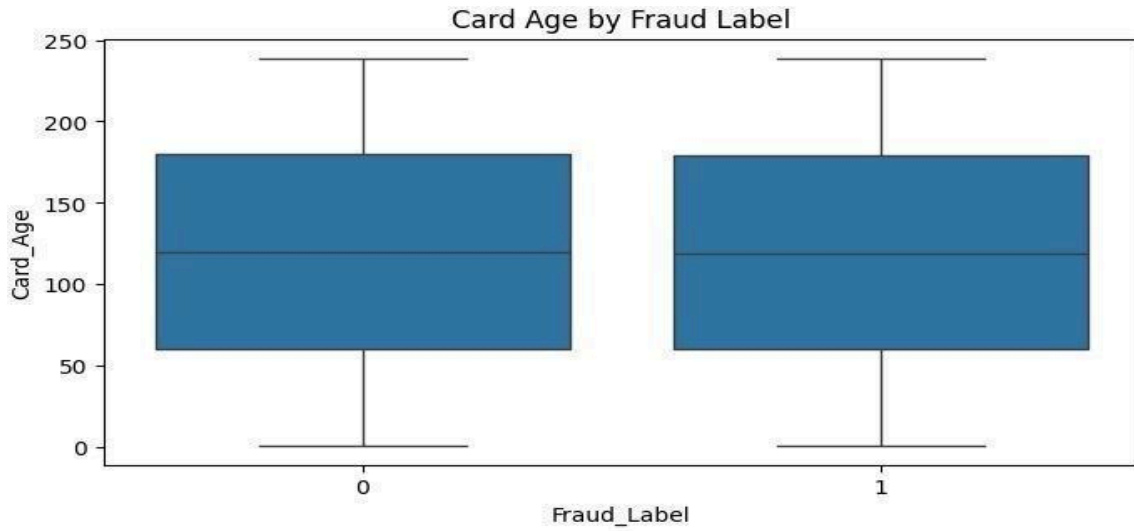


-Average Transaction Amount by Device Type

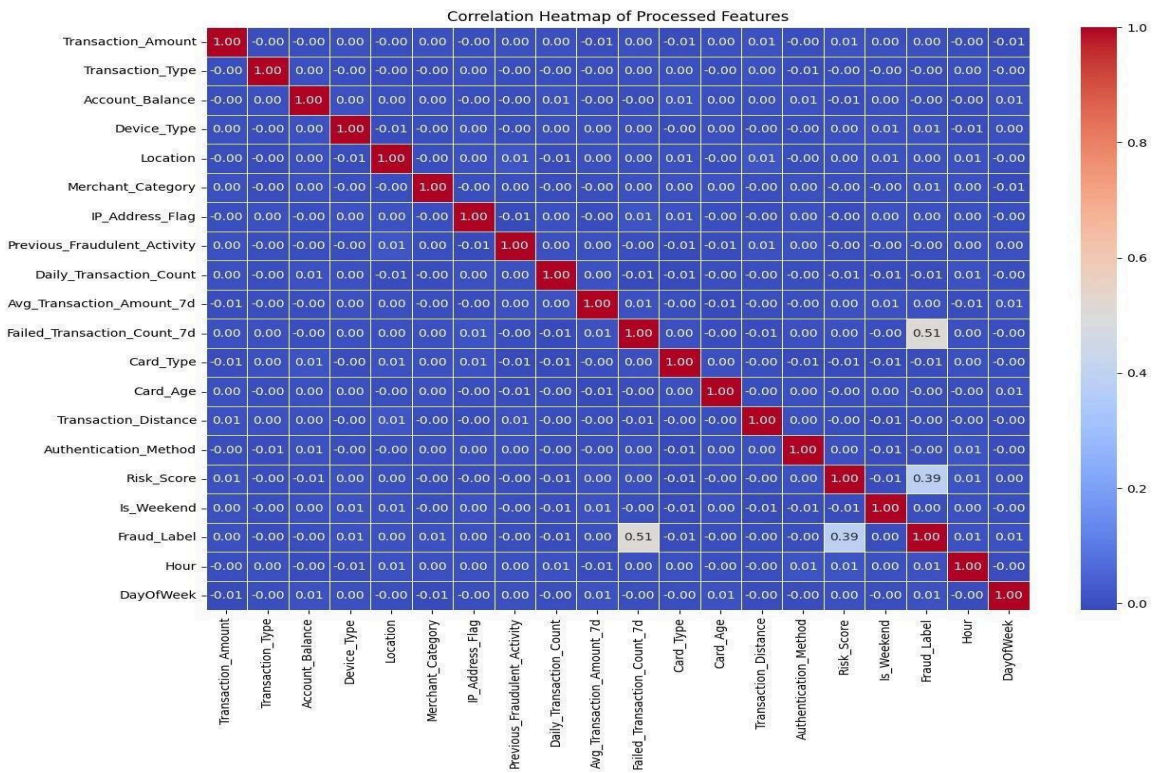


- Risk Score, Transaction Distance, and Card Age by Fraud Label.





_Correlation Heatma



IV. Proposed Model and Evaluation

We implemented two models:

1. Random Forest Classifier: An ensemble model using multiple decision trees. It operates by averaging predictions from multiple trees to reduce overfitting.
2. Linear Regression: A simple regression model, here adapted as a classifier by rounding output predictions to binary values.

Pseudocode for Random Forest:

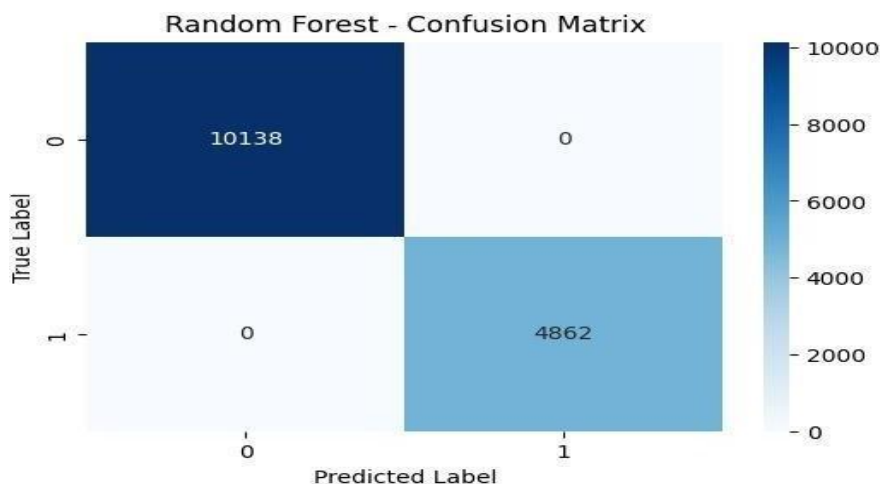
For each tree in the forest:

 Sample data with replacement
 (bootstrap sample) Grow a decision
 tree from the sample

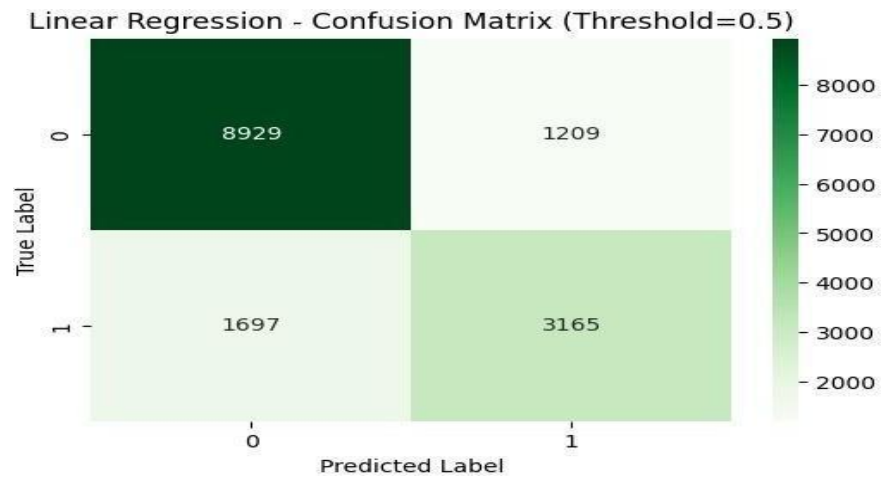
At each node, split based on the best feature among a random subset
Aggregate predictions from all trees for final decision
Evaluation metrics used include Accuracy, Classification Report, Confusion Matrix, Mean Squared Error, and R^2 Score.

Insert here:

-Confusion Matrix for Random Forest



- Confusion Matrix for Linear Regression (as classifier)



V. Results Analysis and Discussion

The Random Forest model achieved high accuracy and demonstrated superior performance in identifying fraudulent transactions compared to the Linear Regression model. Linear Regression showed limited accuracy when used as a classifier. Visualization of confusion matrices and classification reports highlighted these differences. Random Forest is more robust due to its ensemble nature and ability to handle complex data relationships.

The optimal decision is to use Random Forest for fraud detection due to its accuracy and reliability.

VI. Conclusion

This study demonstrated the application of machine learning regression models in fraud detection using a synthetic dataset. Random Forest outperformed Linear Regression, making it a preferable choice for classifying fraudulent transactions. Future work could involve real-time fraud detection systems and incorporating more sophisticated models.

References

- [1] Patil, P. (2024, May 8). Healthcare dataset. Kaggle.
<https://www.kaggle.com/datasets/prasad22/healthcare-dataset>
- [2] Habehh, H., & Gohel, S. (2021). Machine learning in healthcare. *Current Genomics*, 22(4), 291.
- [3] Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- [4] Ng, A. (2020). *Machine Learning Yearning*. DeepLearning.AI.

THE CODE

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    mean_squared_error, r2_score, f1_score, roc_auc_score
)

import warnings

warnings.filterwarnings("ignore")


df = pd.read_csv("synthetic_fraud_dataset.csv")

print(df.info())

print(df.isnull().sum())

df = df.drop_duplicates()


df_processed = df.copy()

df_processed['Timestamp'] =
pd.to_datetime(df_processed['Timestamp'])

df_processed['Hour'] = df_processed['Timestamp'].dt.hour

df_processed['DayOfWeek'] = df_processed['Timestamp'].dt.dayofweek
```

```

categorical_columns = [
    'Transaction_Type', 'Device_Type', 'Location',
    'Merchant_Category', 'Card_Type', 'Authentication_Method'
]

label_encoders = {}

for col in categorical_columns:
    le = LabelEncoder()
    df_processed[col] = le.fit_transform(df_processed[col])
    label_encoders[col] = le

df_processed.drop(['Transaction_ID', 'User_ID', 'Timestamp'],
axis=1, inplace=True)

X = df_processed.drop('Fraud_Label', axis=1)
y = df_processed['Fraud_Label']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

```

```

print("\n[Random Forest Classifier]")

print("Accuracy:", accuracy_score(y_test, y_pred_rf))

print("F1 Score:", f1_score(y_test, y_pred_rf))

print("ROC AUC:", roc_auc_score(y_test,
rf_model.predict_proba(X_test)[: , 1]))

print("\nClassification Report:\n", classification_report(y_test,
y_pred_rf))

```

```

plt.figure(figsize=(6, 4))

sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True,
fmt='d', cmap='Blues')

plt.title("Random Forest - Confusion Matrix")

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.show()

```

```

importances = rf_model.feature_importances_

plt.figure(figsize=(10, 6))

sns.barplot(x=importances, y=X.columns)

plt.title("Feature Importance - Random Forest")

plt.xticks(rotation=45)

plt.show()

```

```

lr_model = LinearRegression()

lr_model.fit(X_train, y_train)

y_pred_reg = lr_model.predict(X_test)

y_pred_class = [1 if val >= 0.5 else 0 for val in y_pred_reg]

```

```

print("\n[Linear Regression as Classifier]")

```

```

print("Accuracy:", accuracy_score(y_test, y_pred_class))

print("F1 Score:", f1_score(y_test, y_pred_class))

print("ROC AUC:", roc_auc_score(y_test, y_pred_reg))

print("\nClassification Report:\n", classification_report(y_test,
y_pred_class))


print("\n[Linear Regression as Regressor]")

print("Mean Squared Error:", mean_squared_error(y_test, y_pred_reg))

print("R^2 Score:", r2_score(y_test, y_pred_reg))


plt.figure(figsize=(6, 4))

sns.heatmap(confusion_matrix(y_test, y_pred_class), annot=True,
fmt='d', cmap='Greens')

plt.title("Linear Regression - Confusion Matrix (Threshold=0.5)")

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.show()


plt.figure(figsize=(6, 4))

sns.countplot(x='Fraud_Label', data=df)

plt.title('Distribution of Fraudulent vs. Non-Fraudulent
Transactions')

plt.xlabel('Fraud Label')

plt.ylabel('Count')

plt.show()


plt.figure(figsize=(8, 5))

sns.boxplot(x='Fraud_Label', y='Transaction_Amount', data=df)

plt.title('Transaction Amount by Fraud Label')

```

```
plt.xlabel('Fraud Label')  
plt.ylabel('Transaction Amount')  
plt.show()
```

```
plt.figure(figsize=(10, 4))  
sns.countplot(x='Transaction_Type', data=df)  
plt.title('Transaction Type Distribution')  
plt.xticks(rotation=45)  
plt.show()
```

```
plt.figure(figsize=(8, 4))  
sns.barplot(x='Device_Type', y='Transaction_Amount', data=df,  
            estimator=np.mean)  
plt.title('Average Transaction Amount by Device Type')  
plt.xticks(rotation=45)  
plt.show()
```

```
plt.figure(figsize=(8, 4))  
sns.boxplot(x='Fraud_Label', y='Risk_Score', data=df)  
plt.title('Risk Score by Fraud Label')  
plt.show()
```

```
plt.figure(figsize=(8, 4))  
sns.boxplot(x='Fraud_Label', y='Transaction_Distance', data=df)  
plt.title('Transaction Distance by Fraud Label')  
plt.show()
```

```
plt.figure(figsize=(8, 4))
```

```
sns.boxplot(x='Fraud_Label', y='Card_Age', data=df)

plt.title('Card Age by Fraud Label')

plt.show()


plt.figure(figsize=(14, 10))

corr_matrix = df_processed.corr()

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm",
            linewidths=0.5)

plt.title("Correlation Heatmap of Processed Features")

plt.show()
```

